

Recurrent Neural Networks

Gunhee Kim



서울대학교
SEOUL NATIONAL UNIVERSITY

Outline

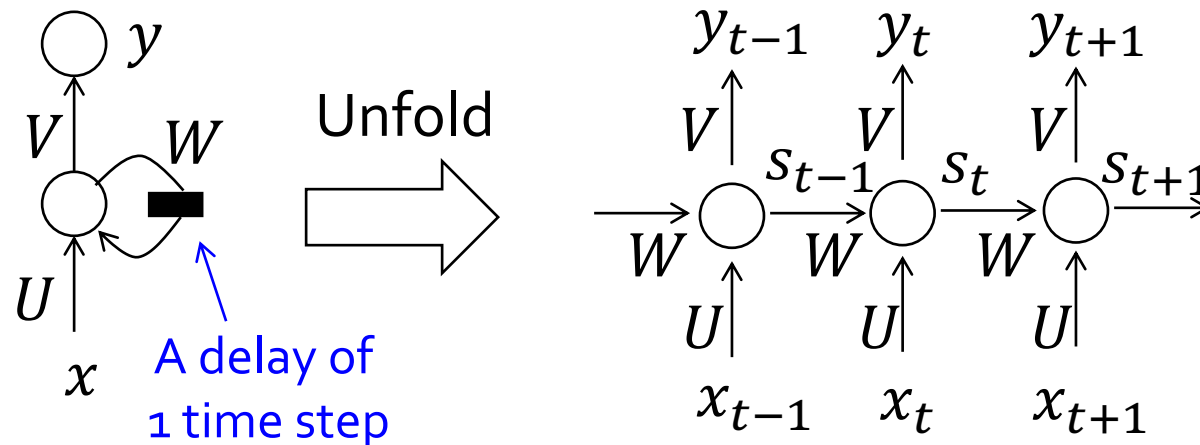
- Recurrent Neural networks
- LSTM Recurrent Networks

Modeling Sequence Data

- Limitation of previous neural networks
 - Input and output are fixed-sized vectors (e.g., an image and probabilities of different classes)
 - A fixed amount of computational steps (e.g., the number of layers in the model)
- Can we handle variable length input and output?
 - Sequence of words in a sentence
 - Sequence of acoustic features at time frames in speech
 - Successive frames in video classification

Recurrent Neural Network

- A generative model for a dynamic system
 - Recurrent: run the same task for every element of a sequence
 - x_t : the input at time step t (e.g., word vector)
 - s_t : the hidden state at time step t
 - y_t : the output at time step t
- Unfolding (unrolling)



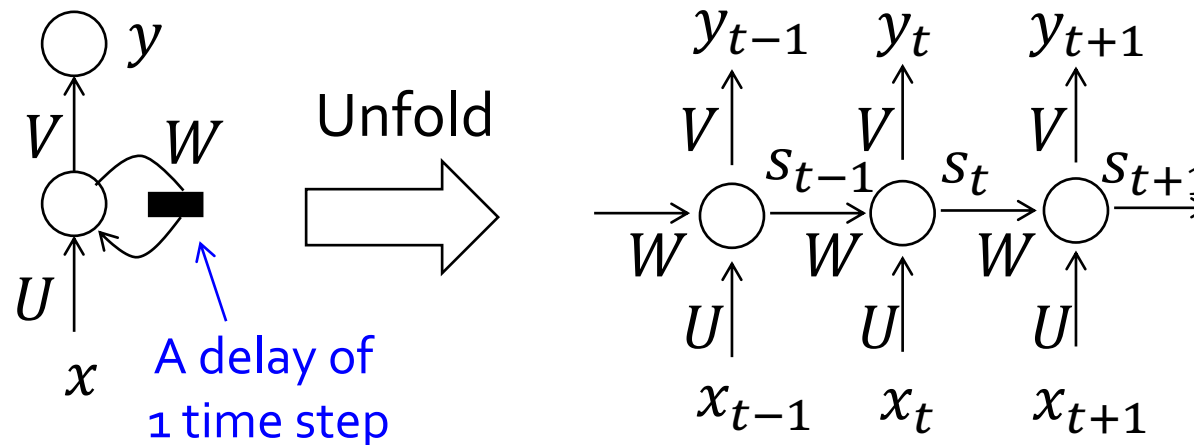
Recurrent Neural Network

- A generative model for a dynamic system
 - The hidden state depends on the previous hidden and input

$$s_t = f(Ux_t + Ws_{t-1} + b)$$

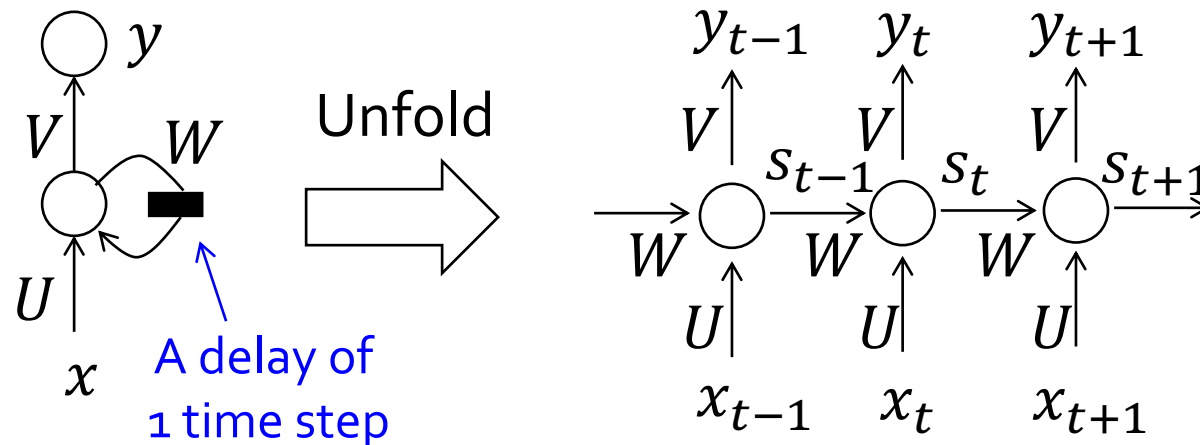
- f is a nonlinear function (hyperbolic tangent (tanh), or ReLU)
- If the output is the prediction of a word

$$y_t = \text{softmax}(Vs_t + c)$$



Recurrent Neural Network

- The hidden state s_t contains information about the whole past sequence
 - Lossy summary from an arbitrary length sequence $x_1, \dots, x_{t-1}$ to a fixed length vector s_t
- RNN shares the same parameters (U, V, W)
 - Perform the same task at each step, just with different inputs
 - Greatly reduces the total number of parameters to learn



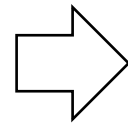
RNN Extension – Bidirectional RNN

- In vanilla RNN, the output at time t only depends on the previous elements in the sequence
 - Let's consider both previous and future elements
 - Combines a forward-going RNN and a backward-going RNN

- Equations

$$s_t = f(Ux_t + Ws_{t-1} + b)$$

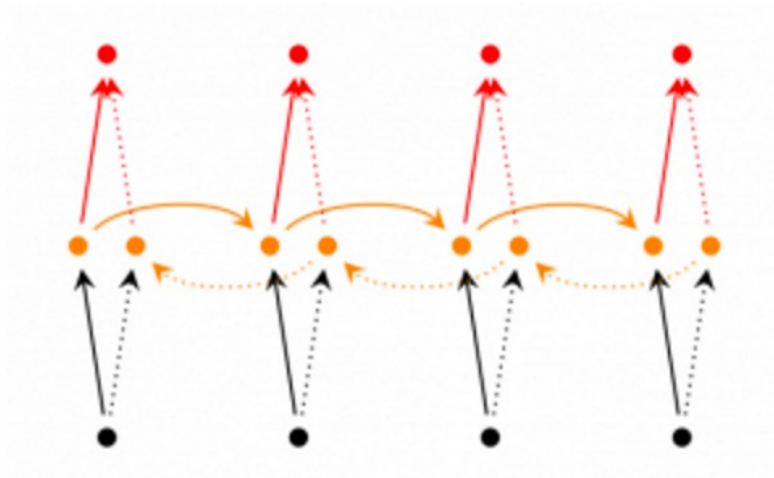
$$y_t = \text{softmax}(Vs_t + c)$$



$$s_t^f = f(U^f x_t + W^f s_{t-1}^f + b^f)$$

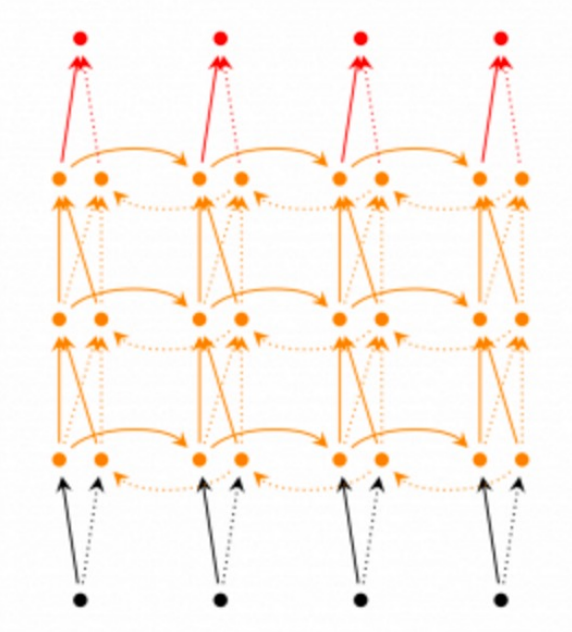
$$s_t^b = f(U^b x_t + W^b s_{t+1}^b + b^b)$$

$$y_t = \text{softmax}(V^f s_t^f + V^b s_t^b + c)$$



RNN Extension – Deep Bidirectional RNN

- Multiple layers per time step
 - Deep in both time and space
 - Higher learning capacity (but need more training data)
- Equations



$$s_t^{f(1)} = f(U^{f(1)}x_t + W^{f(1)}s_{t-1}^{f(1)} + b^{f(1)})$$

$$s_t^{b(1)} = f(U^{b(1)}x_t + W^{b(1)}s_{t+1}^{b(1)} + b^{b(1)})$$

$$s_t^{f(l)} = f(U^{ff(l)}s_t^{f(l-1)} + U^{fb(l)}s_t^{b(l-1)} + W^{f(l)}s_{t-1}^{f(l)} + b^{f(l)})$$

$$s_t^{b(l)} = f(U^{bf(l)}s_t^{f(l-1)} + U^{bb(l)}s_t^{b(l-1)} + W^{b(l)}s_{t-1}^{b(l)} + b^{b(l)})$$

$$y_t = \text{softmax}(V^{f(L)}s_t^{f(L)} + V^{b(L)}s_t^{b(L)} + c)$$

BPTT Algorithm

- BackPropagation Through Time
 - One of the methods used to train RNNs
 - The unfolded network is treated as one big feed-forward network
 - This big network takes in entire sequence as an input
 - Compute gradients through the usual backpropagation
 - Update shared weights

Vanishing and (Exploding) Gradient Problem

- RNNs have difficulties learning long-range dependencies
- In very deep nets and recurrent nets, the final output is composed of many non-linear transformations
 - Although each non-linear state may be smooth enough, its composition is going to be very steep
 - Multiplying the same value many times tends to be either very large or very small
 - Gradients become either very small or very large



Vanishing and (Exploding) Gradient Problem

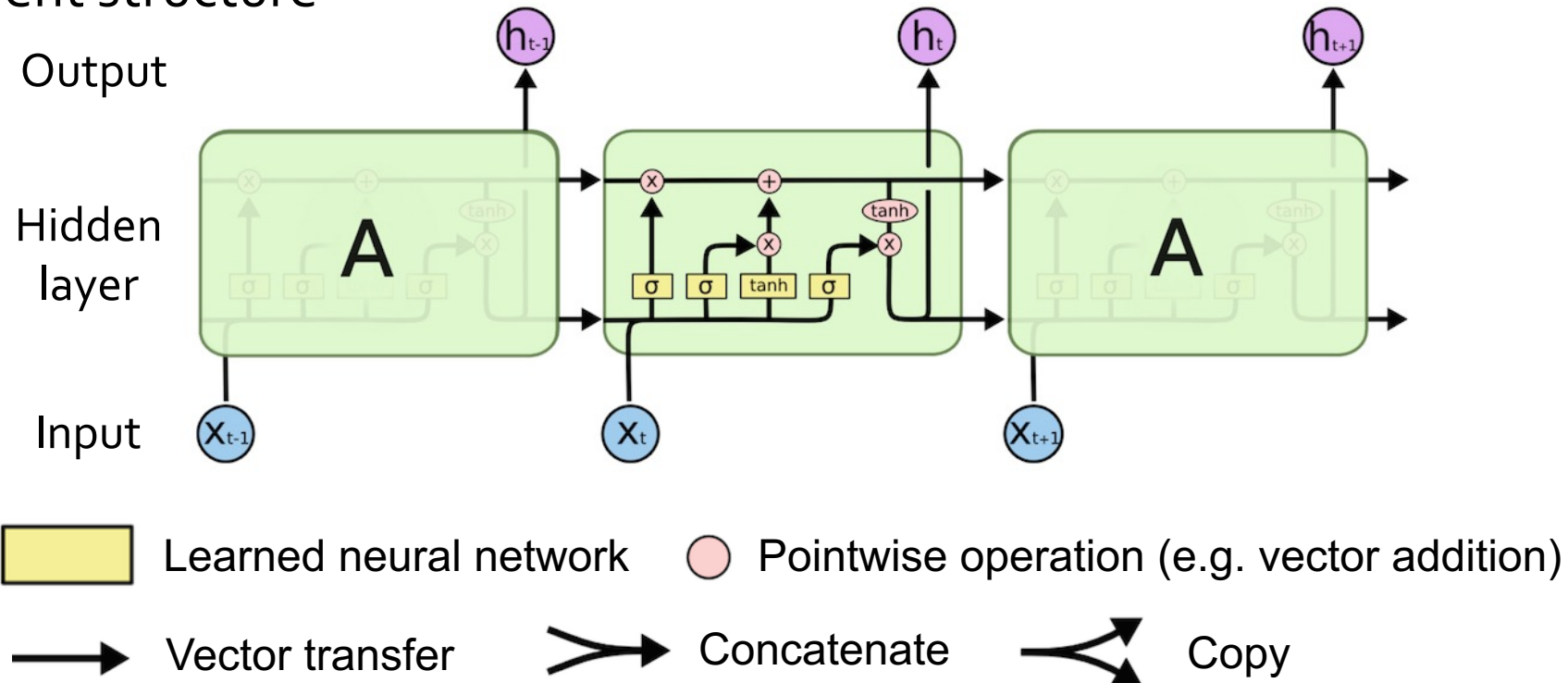
- Exploding gradient is also problematic but get less attention
 - Easy to detection (e.g., finding NaN)
 - Easy to remedy (e.g., using *clipping* by a pre-defined threshold)
- Use Long Short-Term Memory (LSTM)
 - Same architecture with RNN but different functions for hidden states
 - Design a cell and gates to keep in memory
- Transformers are popular these days!
 - e.g., BERT, GPT-3 to name a few

Outline

- Recurrent Neural networks
- LSTM Recurrent Networks

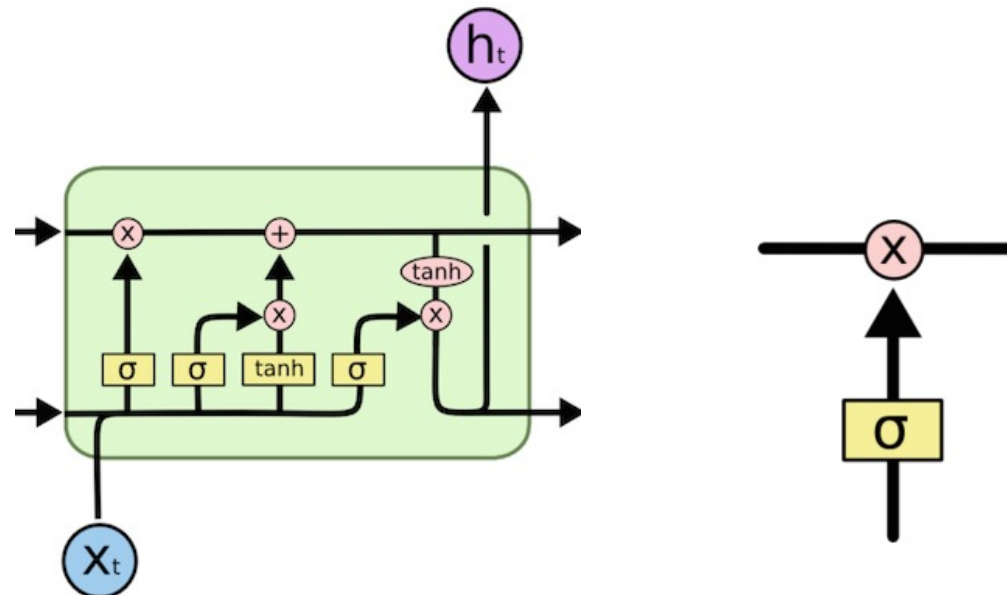
LSTM Networks

- Designed to avoid the long-term dependency problem
 - How? We will see it later
 - Have the same chain structure with RNNs, but the repeating hidden module has different structure



LSTM Networks

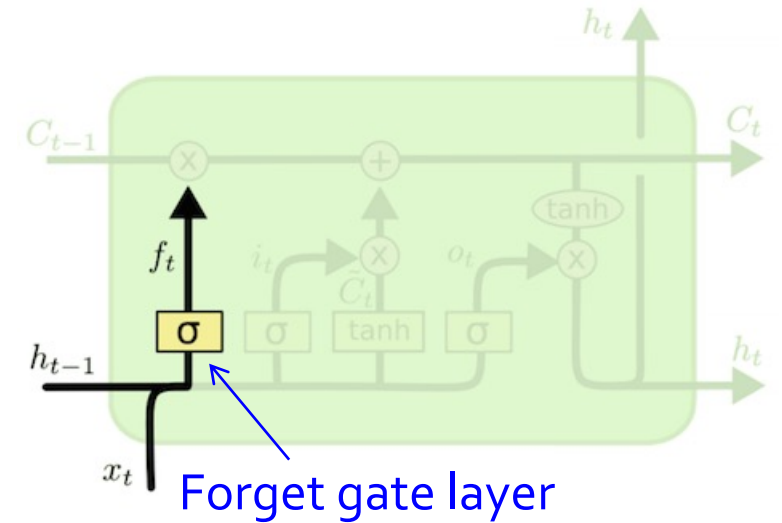
- A cell and three gates in an LSTM unit
 - Each gate protects and controls the cell state
 - Each gate regulates the information flow
 - Consists of a sigmoid (or tanh) layer and pointwise multiplication
 - Sigmoid output ranges $[0, 1]$ (turn off/on)



Step-by-Step LSTM Walk Through

- First step
 - Look at previous output h_{t-1} and current input x_t
 - Decide whether previous cell state C_{t-1} is forgotten or not

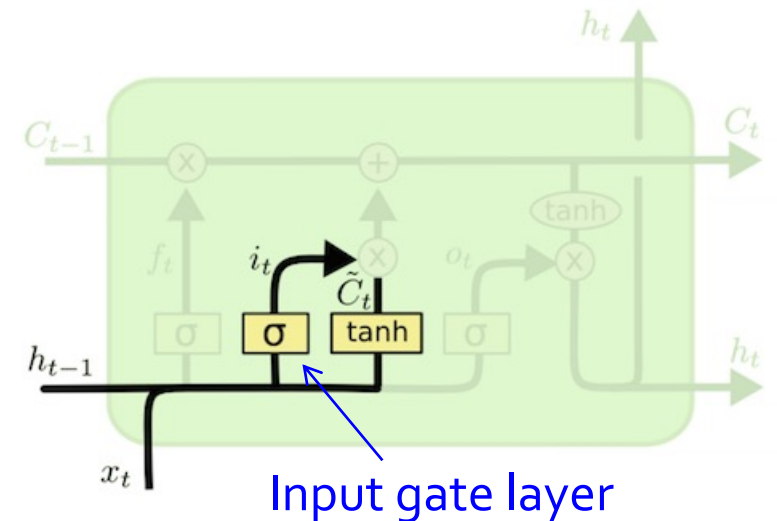
$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$



- Second step
 - The sigmoid layer decides which values are updated
 - The tanh layer creates new values to be added to the state

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$



Step-by-Step LSTM Walk Through

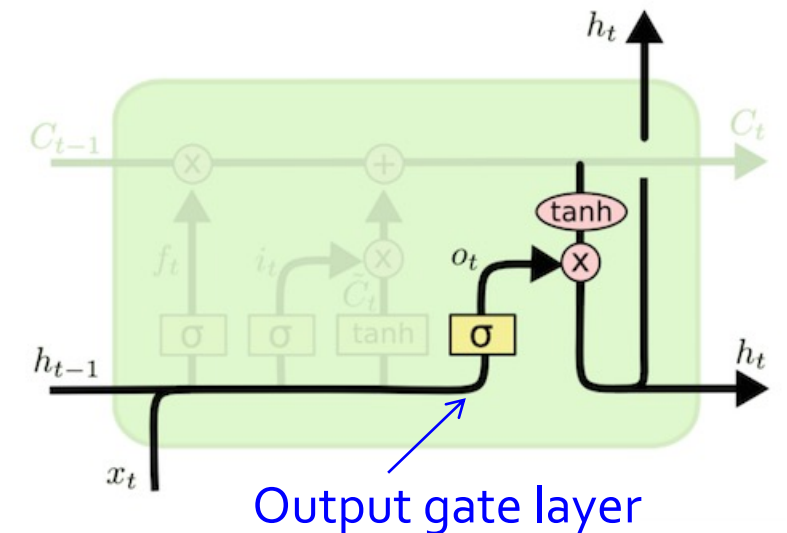
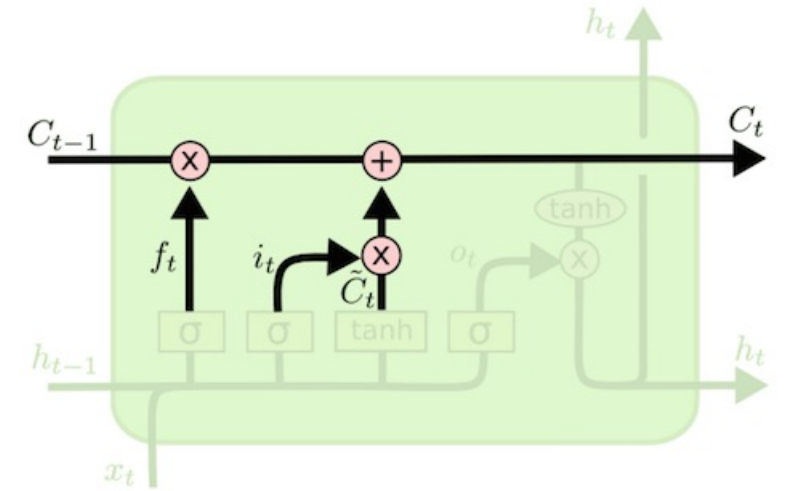
- Third step
 - Old state C_{t-1} multiplied by f_t (forgetting factor)
 - New candidate values multiplied by i_t (update factor)

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

- Fourth step
 - The sigmoid works like a filter
 - New cell state C_t goes through tanh (push values within $[-1,1]$)

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t)$$



Summary of LSTM Networks

- LSTM cell can be defined with following equations

- Gates (input / forget / output)

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

- Input transform

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

- State update

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

$$h_t = o_t * \tanh(C_t)$$

How LSTM Handles Long-Term Dependency

- Networks can retain/discard the information for long time
 - We can regulate f_t, i_t to control how much previous cell and new input information

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

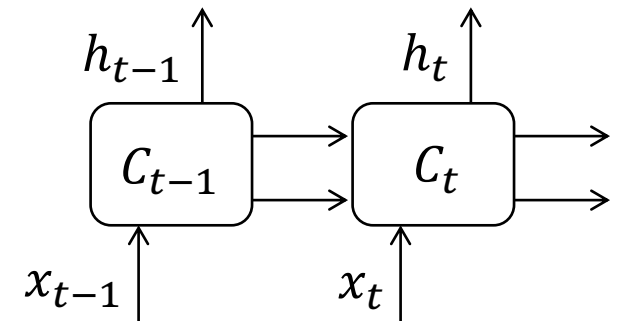
- Prevent vanishing/exploding gradient problem

- Suppose the error at t is $\frac{\partial L_t}{\partial C_t}$

- By chain rule, $\frac{\partial L_t}{\partial C_{t-1}} = \frac{\partial L_t}{\partial C_t} \frac{\partial C_t}{\partial C_{t-1}} = f_t * \frac{\partial L_t}{\partial C_t}$

$$\frac{\partial L}{\partial C_{t-1}} = \frac{\partial L_{t-1}}{\partial C_{t-1}} + f_t * \frac{\partial L_t}{\partial C_t}$$

- If $f_t \sim 1$, it is not vanishing; since $f_t < 1$, it is not exploding



Variants on LSTM

- Add peepholes to all the gates
 - Let all gates look at the cell states

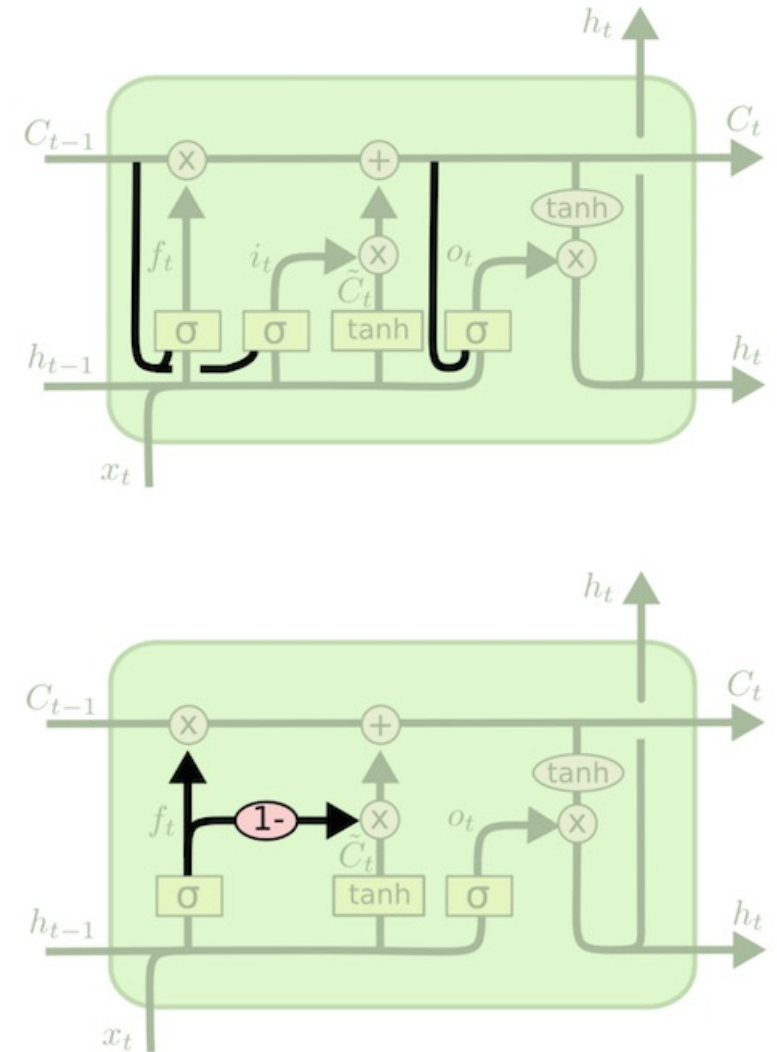
$$f_t = \sigma(W_f \cdot [C_{t-1}, h_{t-1}, x_t] + b_f)$$

$$i_t = \sigma(W_i \cdot [C_{t-1}, h_{t-1}, x_t] + b_i)$$

$$o_t = \sigma(W_o \cdot [C_{t-1}, h_{t-1}, x_t] + b_o)$$

- Couple forget and input gates
 - Replace input gate by $(1 - \text{forget})$

$$C_t = f_t * C_{t-1} + (1 - f_t) * \tilde{C}_t$$



Variants on LSTM

- GRU (Gated Recurrent Unit)
 - Much simpler than LSTM
 - Merges the cell state and hidden state
 - Combines the forget and input gates into a single *update gate*
 - And other minor modifications

$$z_t = \sigma(W_z \cdot [h_{t-1}, h_t])$$

$$r_t = \sigma(W_r \cdot [h_{t-1}, h_t])$$

$$\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t])$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

